

Software Engineering Models and Methods

ACRONYMS

3GL	3rd Generation Language
BNF	Backus-Naur Form
FDD	Feature-Driven Development
IDE	Integrated Development Environment
RAD	Rapid Application Development
UML	Unified Modeling Language
XP	eXtreme Programming

INTRODUCTION

Software engineering models and methods impose structure on software engineering to make it systematic, repeatable and ultimately more success-oriented. Models provide an approach to problem-solving, a notation and procedures for model construction and analysis. Methods provide an approach to the systematic specification, design, construction, testing and verification of the end-item software and associated work products.

Software engineering models and methods vary widely in scope — from addressing a single software life cycle phase to covering the complete software life cycle. This knowledge area (KA) focuses on models and methods that encompass multiple software life cycle phases regardless of the type of life cycle process models such as iterative models and agile ones, since other KAs cover methods specific to single life cycle phases.

BREAKDOWN OF TOPICS FOR SOFTWARE ENGINEERING MODELS AND METHODS

This chapter on software engineering models and methods is divided into four main topic areas:

1. *Modeling* discusses the general practice of modeling and presents topics in modeling principles, properties and expression of models, modeling syntax, semantics, and pragmatics, as well as preconditions, postconditions, and invariants.
2. *Types of Models* briefly discusses models and aggregation of submodels and provides general characteristics of model types commonly found in the software engineering practice.
3. *Analysis of Models* presents common analysis techniques used in modeling to verify completeness, consistency, correctness, traceability and interaction.
4. *Software Engineering Methods* presents a summary of commonly used software engineering methods, such as heuristic methods, formal methods, prototyping and Agile methods.

The breakdown of topics for the Software Engineering Models and Methods KA is shown in Figure 11.1.

1. Modeling

Modeling of software is a pervasive technique to help software engineers understand, engineer and communicate aspects of the software to appropriate stakeholders. *Stakeholders* are those people or parties with a stated or implied

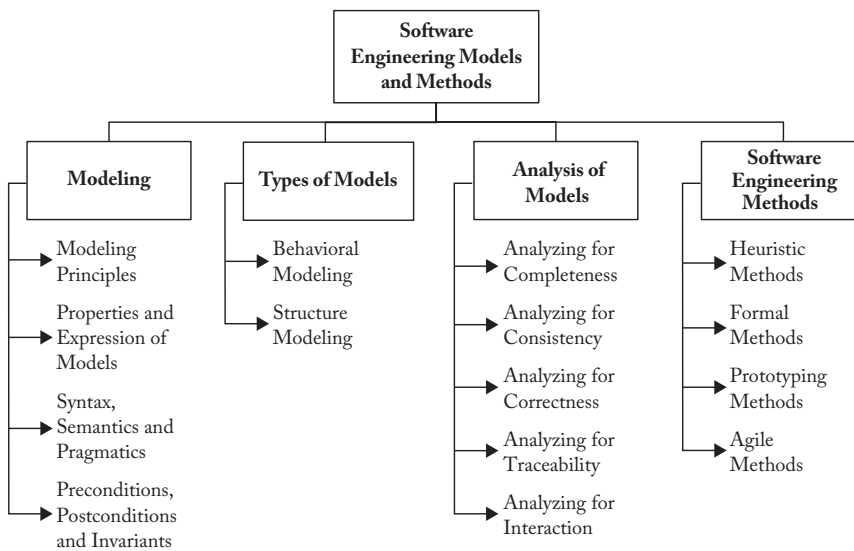


Figure 11.1. Breakdown of Topics for the Software Engineering Models and Methods KA

interest in the software (e.g., users, buyers, suppliers, architects, certifying authorities, evaluators, developers, software engineers).

Although there are many modeling languages, notations, techniques and tools in the literature and in practice, some general, unifying concepts apply to them all. The following sections provide background on these general concepts.

1.1. Modeling Principles

[1*, c2s2, c5s1, c5s2, 2*, c2s2, 3*, c5s0]

Modeling provides the software engineer with an organized and systematic approach for representing significant aspects of the software under study, facilitating decision-making about the software or elements, and communicating those significant decisions to others in the stakeholder communities. Three general principles guide such modeling activities:

- *Model the essentials:* Good models do not represent every aspect or feature of the software under every possible condition. Modeling involves only those aspects

or features that pose specific questions, abstracting away any nonessential information. This approach keeps models manageable and useful.

- *Provide perspective:* Modeling provides views of the software under study using defined rules for expressing the model within each view. This perspective-driven approach provides dimensionality to the model (e.g., providing a structural view, a behavioral view, a temporal view, an organizational view and/or other views if relevant). Organizing information into views focuses the software modeling efforts on specific concerns relevant to that view using the appropriate notation, vocabulary, methods and tools.
- *Enable effective communications:* Modeling uses the application domain vocabulary of the software, a modeling language and semantic expression (in other words, meaning within context). When used rigorously and systematically, modeling results in a reporting approach that facilitates effective communication of software information to project stakeholders.

A model is an *abstraction* or simplification of a system. A consequence of using abstraction is that, because no single abstraction completely describes a software component, the software model comprises an aggregation of abstractions, which, when taken together, describe selected aspects, perspectives or views — only those that are needed to make informed decisions and respond to the reasons for creating the model in the first place. This simplification points to assumptions about the context within which the model is placed that should also be captured in the model. Then, when the model is reused, these assumptions can be validated first to establish the relevancy of the reused model within its new use and context.

1.2. Properties and Expression of Models

[1*, c5s2, c5s3, 3*, c4s1.1p7, c4s6p3, c5s0p3]

Properties of models are those distinguishing features of a particular model that characterize its completeness, consistency and correctness within the chosen modeling notation and tooling. Properties of models include the following:

- *Completeness* — the degree to which all requirements have been implemented and verified within the model
- *Consistency* — the degree to which the model contains no conflicting requirements, assertions, constraints, functions or component descriptions
- *Correctness* — the degree to which the model satisfies its requirements and design specifications and is free of defects

Models are constructed to represent objects needed for target domains and their behaviors to answer specific questions about how the software is expected to operate. Interrogating the models — through exploration, simulation or review — might expose areas of uncertainty within the model and the software to which the model refers. These uncertainties or unanswered questions regarding the

requirements, design and/or implementation can then be handled appropriately.

The primary expression element of a model is an *entity*. An entity may represent concrete artifacts (e.g., processors, sensors or robots) or abstract artifacts (e.g., software modules or communication protocols). Model entities are connected to other entities using relations (lines or textual operators on target entities). Expression of model entities may be accomplished using textual or graphical modeling languages; both modeling language types connect model entities through specific language constructs. The meaning of an entity may be represented by its shape, its textual attributes or both. Generally, textual information adheres to language-specific syntactic structure. The precise meanings related to the modeling of context, structure or behavior using these entities and relations are dependent on the modeling language used, the design rigor applied to the modeling effort, the specific view being constructed and the entity to which the specific notation element may be attached. Multiple views of the model may be required to capture the needed semantics of the software.

When using automation-supported models, models may be checked for completeness and consistency. The usefulness of these checks depends greatly on the level of semantic and syntactic rigor applied to the modeling effort and on explicit tool support. Correctness can be checked through model simulation, execution or review.

1.3. Syntax, Semantics, and Pragmatics

[2*, c2s2.2.2p6, 3*, c5s0]

Models can be surprisingly deceptive. The fact that a model is an abstraction with missing information can give people the illusion that they completely understand the software after studying a single model. A *complete model* (“complete” being relative to the modeling effort) may be a union of multiple submodels and any special function models. Examination of and decision-making regarding a single model within this collection of submodels may be problematic.

Understanding the precise meanings of modeling constructs can also be difficult. Syntactic and semantic rules define modeling languages. For textual languages, *syntax* is defined using a notation grammar that defines valid language constructs (e.g., Backus-Naur form (BNF)). For graphical languages, syntax is defined using graphical models called *meta-models*. As with BNF, metamodels define a graphical modeling language's valid syntactical constructs. In addition, the metamodel defines how these constructs can be composed to produce valid models.

Semantics for modeling languages specify the meaning attached to the entities and relations captured within the model. For example, a simple diagram of two boxes connected by a line is open to various interpretations. Knowing that the diagram on which the boxes are placed and connected is an object diagram or an activity diagram assists in interpreting this model.

As a practical matter, the semantics of a specific software model are usually fairly clear due to the model's use of a modeling language, the way that modeling language expresses entities and relations within that model, the experience and skill of the modelers, and the context within which the modeling has been undertaken and represented. Meaning is communicated through the model even in the presence of incomplete information through abstraction. *Pragmatics* explains how meaning is embodied in the model and its context and how it is communicated effectively to other software engineers.

However, there are still instances where caution is needed regarding modeling and semantics. For example, any model parts imported from another model or library must be examined for semantic assumptions that conflict with the new modeling environment; these conflicts might not be obvious. The model should be checked for documented assumptions. Although the imported modeling syntax might be the same, it might mean something quite different in the new environment, which is a different context. Also, consider that as software matures and changes

are made, semantic discord can be introduced, leading to errors. With many software engineers working on part of a model over time, and with tool updates and perhaps new requirements, it is possible for portions of the model to represent something different from the original author's intent and initial model context.

1.4. *Preconditions, Postconditions, and Invariants* [2*, c4s4, 4*, c10s4p2, c10s5p2p4]

When modeling functions or methods, the software engineer typically starts with assumptions about the software's state before, during and after the function or method executes. These assumptions are essential to the correct operation of the function or method and are grouped, for discussion, as a set of preconditions, postconditions and invariants.

- *Preconditions* are conditions that must be satisfied before execution of the function or method. If these preconditions do not hold before execution of the function or method, the function or method might produce erroneous results.
- *Postconditions* are conditions guaranteed to be true after the function or method has executed successfully. Typically, the postconditions represent how the software's state has changed, how parameters passed to the function or method have changed, how data values have changed, or how the return value has been affected.
- *Invariants* are conditions within the operational environment that persist (in other words, do not change) before and after execution of the function or method. These invariants are relevant and necessary to the software and to the correct operation of the function or method.

2. Types of Models

A typical model consists of an aggregation of submodels. Each submodel is a partial description and is created for a specific purpose. A submodel may comprise one or more

diagrams. The collection of submodels may use multiple modeling languages or a single modeling language. The unified modeling language (UML) recognizes a rich collection of modeling diagrams. These diagrams, along with the modeling language constructs, are used in two common model types: structural models and behavioral models. (See Section 1.1.) Depending on modeling languages, there can be other types of models. For instance, the systems modeling language (SysML) provides two other types of models: requirements models and parametric models.

2.1. Structural Modeling

[1*, c7s2.2, c7s2.5, c7s3.1, c7s3.2, 3*, c5s3, c8s1, 4*, c4, 17]

Structural models illustrate the software's physical or logical composition of software from its various component parts. *Structural modeling* establishes the defined boundary between the software being implemented or modeled and the environment in which it is to operate. Some common structural constructs used in structural modeling are composition, decomposition, generalization, and specialization of entities; identification of relevant relations and cardinality between entities; and the definition of process or functional interfaces. Structure diagrams provided by the UML for structural modeling include class, component, object, deployment, and packaging diagrams.

Information modeling is a kind of structural modeling and focuses on data and other information. An *information model* is an abstract representation that identifies and defines a set of concepts, properties, relations and constraints on data entities. The semantic or conceptual information model is often used to provide some formalism and context to the software as viewed from the problem perspective, without concern for how this model is mapped to the implementation of the software. The semantic or conceptual information model is an abstraction and, as such, includes only the concepts, properties, relations and constraints needed to conceptualize a real-world view of

the information. Subsequent transformations of the semantic or conceptual information model become logical and then physical data models as implemented in the software.

2.2. Behavioral Modeling

[1*, c7s2.1, c7s2.3, c7s2.4, 2*, c9s2, 3*, c5s4, 8, c1s5.4]

Behavioral models identify and define software functions. Behavioral models generally take three basic forms: state machines, control-flow models and data-flow models. State machines provide a model that represents the software as a collection of defined states, events and transitions. The software transitions from one state to the next through a guarded or unguarded triggering event that occurs in the modeled environment. Control-flow models depict how a sequence of events causes processes to be activated or deactivated. Data-flow models represent data-flow behavior as a sequence of steps where data moves through processes toward data stores or data sinks. These models are described in the way of event-triggered, time concepts (i.e., logical, physical, discrete, continuous, relative, or absolute time), or combinations thereof. Behavioral diagrams provided by the UML for behavioral modeling include use case, activity, state machine, and interaction (sequence, communication, timing, and interaction overview) diagrams.

3. Analysis of Models

The development of models allows the software engineer to study, reason about and understand software structure, function, operational use and assembly considerations. Analysis of constructed models is needed to ensure that the models are complete, consistent and correct enough to serve their intended purpose for the stakeholders.

The following sections briefly describe the analysis techniques generally used to ensure that the software engineer and other relevant stakeholders gain appropriate value from the development and use of models.

3.1. Analyzing for Completeness

[3*, c4s1.1p7, c4s6, 5*, pp8-11]

To ensure software fully meets the needs of the stakeholders, completeness — from the requirements elicitation process to code implementation — is critical. *Completeness* is the degree to which all specified requirements have been implemented and verified. Engineers can check models for completeness with a modeling tool that uses structural analysis and state-space reachability analysis (which ensure some set of correct inputs reach all paths in the state models). Models may also be checked manually for completeness by using inspections or other review techniques. (See the Software Quality KA.) Errors and warnings generated by these analysis tools and found by inspection or review indicate that corrective actions are probably needed to ensure model completeness.

3.2. Analyzing for Consistency

[3*, c4s1.1p7, c4s6, 5*, pp8-11]

Consistency is the degree to which models contain no conflicting requirements, assertions, constraints, functions or component descriptions. Typically, consistency checking is accomplished with the modeling tool using an automated analysis function. Models may also be checked manually for consistency using inspections or other review techniques. (See the Software Quality KA.) As with completeness, errors and warnings generated by these analysis tools and found by inspection or review indicate the need for corrective action.

3.3. Analyzing for Correctness [5*, pp8-11]

Correctness is the degree to which a model satisfies its software requirements and software design specifications, is free of defects, and ultimately meets the stakeholders' needs. Analyzing for correctness includes verifying the model's syntactic correctness (that is, correct use of the modeling language grammar and constructs) and semantic correctness (that is, use of the modeling language constructs to correctly represent the meaning of that which

is being modeled). To analyze a model for syntactic and semantic correctness, one analyzes it — either automatically (e.g., using a modeling tool to check for model syntactic correctness) or manually (using inspections or other review techniques) — searching for possible defects and then removing or repairing the confirmed defects before the software is released for use.

3.4. Analyzing for Traceability

[3*, c4s7.1, c4s7.2]

Developing software typically involves using, creating and modifying many work products such as planning documents, process specifications, software requirements, diagrams, designs and pseudo-code, handwritten and tool-generated code, manual and automated test cases and reports, and files and data. These work products may share various dependency relationships (e.g., uses, implements and tests). As software is developed, managed, maintained or extended, these traceability relationships must be mapped and controlled to demonstrate the software requirements' consistency with the software model (see Requirements Tracing in the Software Requirements KA) and the many work products. Use of traceability typically improves the management of software work products and software process quality and assures stakeholders that all requirements are satisfied. Traceability enables change analysis once the software is developed and released because relationships to software work products can easily be traversed to assess change impact. Modeling tools typically help automatically or manually specify and manage traceability links among requirements, design, code and/or test entities that might be represented in the models and other software work products. (For more information on traceability, see the Software Configuration Management KA.)

3.5. Analyzing for Interaction

[2*, c10, c11, 3*, c29s1.1, c29s5, 4*, c5]

Interaction analysis focuses on the communications or control-flow relations between entities used to accomplish a specific task or function

within the software model. This analysis examines the dynamic behavior of the interactions among the software model's different parts, including other software layers (such as the operating system, middleware and applications). Examining interactions between the computer software application and the user interface software is also important for some software applications. Some software modeling environments provide simulation facilities to study aspects of the dynamic behavior of modeled software. Stepping through the simulation allows the software engineer to review the interaction design and verify that the software's different parts work together to provide the intended functions.

4. Software Engineering Methods

Software engineering methods provide an organized and systematic approach to developing software for a target computer. There are numerous methods from which to choose, and the software engineer needs to choose an appropriate method or methods for the software development task at hand. This choice can dramatically affect the success of the project. When software engineers, working with people who have the right skill sets and the right tools, use these software engineering methods, they can visualize the software's details and ultimately transform the representation into a working set of code and data.

Selected software engineering methods are discussed below. The topic areas are organized into discussions of Heuristic Methods, Formal Methods, Prototyping Methods and Agile Methods.

4.1. Heuristic Methods [1*, c13, c15, c16, 3*, c2s2.2, c7s1, c5, 8, pp.xiii-xvii 9, c2s2, 11, c1, 12, c1s1, 19, pp.220-242]

Heuristic methods are experience-based software engineering methods, which are fairly widely practiced in the software industry. This topic area contains five broad discussion categories: structured analysis and design methods, data modeling methods, object-oriented

analysis and design methods, aspect-oriented development methods, and model-driven and model-based development methods.

- *Structured analysis and design methods:* These methods develop the software model primarily from a functional or behavioral viewpoint. They start from a high-level view of the software (including data and control elements). It then progressively decomposes or refines the model components through increasingly detailed designs. The detailed designs eventually converge to specific software details or specifications that must be coded (by hand, automatically generated or both), built, tested and verified.
- *Data modeling methods:* The data model is constructed from the viewpoint of the data or information used. Data tables and relationships define the data models. This data modeling method is used primarily to define and analyze data requirements supporting database designs or data repositories typically found in business software, where data is actively managed as a business systems resource or asset.
- *Object-oriented analysis and design methods:* The object-oriented model is represented as a collection of objects that encapsulate data and relationships and interact with other objects through methods. Objects may be real-world items or virtual items. These methods build models using diagrams to constitute selected views of the software. Progressive refinement of the models leads to a detailed design. The detailed design is then either evolved through successive iterations or transformed (using some mechanism) into the implementation view of the model, where the code and packaging for eventual software product release and deployment are expressed. Popular object-oriented approaches include Unified Process (UP) and specific implementations of UP, such as Rational Unified Process (RUP). (See the section Model-Based Requirements in the Software Design KA and Software Requirements KA.)

- *Aspect-Oriented Development Methods:* The aspect-oriented approach aims to separate crosscutting concerns from non-crosscutting ones in the system and keeps them encapsulated throughout the entire life cycle to solve their scattering and tangling problem. Aspect is the unit of modularity to encapsulate crosscutting concerns. At the software level, there is a “weaver” that is in charge of joining the portions of functionality (advices) encapsulated in the incumbencies at certain points of base behavior (join points), according to well-defined predicates (pointcuts).
- *Model-Driven and Model-Based Development Methods:* Model-Driven Development (MDD) is an approach using models as primary artifacts of the development process. In MDD, usually the implementation or other models are (semi)automatically transformed from the models. Model-Based Development (MBD) uses models to analyze the system, where models are not necessarily the primary artifacts. Some literature refers to MBD as the acronym for Model-Based Design. Model-Based Design is a model-centric approach to developing control, signal processing, communications, and other dynamic systems, focusing on executable specification and simulation. See the Software Design KA. Model-Driven Requirements and Model-Based Requirements apply the same mentality to specification of software requirements, see the Software Requirements KA. MDD/MBD is a prerequisite to Model-Based Architecture, see the Software Architecture KA. Sometimes, test cases are generated from models, see the Software Testing KA.

4.2. Formal Methods

[1*, c18, 3*, c27, 5*, pp8-24, 10, pp.xi-xiv]

Formal methods are software engineering methods that apply rigorous, mathematically based notation and language to specify, develop and verify the software. Through

use of a specification language, the software model can be systematically checked for consistency (or lack of ambiguity), completeness, and correctness, either automatically or semiautomatically. This topic is related to the Formal Analysis section in the Software Requirements KA.

This section addresses specification languages, program refinement and derivation, formal verification, logical inference, and lightweight formal methods.

- *Specification languages:* Specification languages provide the mathematical basis for a formal method. Specification languages are formal, higher-level computer languages (not a classic 3rd-generation language (3GL) programming language) used during the software specification, requirements analysis and/or design stages to describe specific input/output behavior. Specification languages are not directly executable languages. Instead, they typically comprise a notation and syntax, semantics for the use of the notation, and a set of allowed relations for objects.
- *Program refinement and derivation:* Program refinement creates a lower-level (or more detailed) specification using a series of transformations. Through successive transformations, the software engineer derives an executable representation of a program. Specifications may be refined, adding details until the model can be formulated in a 3GL programming language or in an executable portion of the chosen specification language. This specification refinement is made possible by defining specifications with precise semantic properties. For example, the specifications must set out not only the relationships between entities but also the exact runtime meanings of those relationships and operations.
- *Formal verification:* Model checking is a formal verification method. It typically involves performing a state-space exploration or reachability analysis to demonstrate that the represented software design

has or preserves certain model properties of interest. An example of model checking is an analysis that verifies correct program behavior under all possible interleaving of event or message arrivals. Formal verification requires a rigorously specified model of the software and its operational environment. This model often takes the form of a finite-state machine or other formally defined automaton.

- *Logical inference*: Logical inference is a method of designing software that specifies preconditions and postconditions around each significant design block. Using mathematical logic, it develops the proof that those preconditions and postconditions must hold under all inputs. This allows the software engineer to predict software behavior without having to execute the software. Some integrated development environments (IDEs) include ways to represent these proofs and the design or code.
- *Lightweight Formal Methods*: Lightweight formal methods are lightweight approaches that balance practical usability and rigorous verification. For instance, Alloy takes from formal specification the idea of a precise and expressive notation based on a tiny core of simple and robust concepts, but it replaces conventional analysis based on theorem proving with a fully automatic analysis that gives immediate feedback. Unlike theorem proving, this analysis is not “complete”: it examines only a finite space of cases.

4.3. Prototyping Methods

[1*, c12s2, 3*, c2s3.1, 6*, c7s3p5]

Software *prototyping* is an activity that generally creates incomplete or minimally functional versions of a software application, usually for trying out specific new features; soliciting feedback on software requirements or user interfaces; further exploring software requirements, software design, or implementation options; or gaining some other useful insight into the software. The software

engineer selects a prototyping method to first understand the least understood software aspects or components. This approach contrasts with other software engineering methods that usually begin development with the best-understood portions first. Typically, the prototype does not become the final software product without extensive development rework or refactoring.

This section briefly discusses prototyping styles, targets and evaluation techniques.

- *Prototyping style*: Prototyping styles describe the various approaches to developing prototypes. A prototype can be developed as throwaway code or a paper product, as an evolution of a working design, or as an executable specification. Different prototyping life cycle processes are typically used for each style. The style chosen is based on the type of results the project needs, the quality of the results needed and the results’ urgency.
- *Prototyping target*: The prototyping target is the specific product served by the prototyping effort. Examples of prototyping targets are a requirements specification, an architectural design element or component, an algorithm, and a human-machine user interface.
- *Prototyping evaluation techniques*: The software engineer or other project stakeholders may use or evaluate the prototype in many ways, driven primarily by the underlying reasons that led to prototype development. Prototypes may be evaluated or tested against the implemented software or target requirements (e.g., a requirements prototype). The prototype might also serve as a model for future software development (e.g., as in a user interface specification).

4.4. Agile Methods

[3*, c3, 6*, c7s3p7, 7*, c6, App. A, 13, 14, 15, 16, 18]

Agile methods were developed in the 1990s to reduce the apparent large overhead associated

with heavyweight, plan-based methods used in large-scale software development projects. Agile methods are considered lightweight because of their short, iterative development cycles, self-organizing teams, simpler designs, code refactoring, test-driven development, frequent customer involvement and emphasis on creating a demonstrable working product with each development cycle. Agile methods can be seen as an application of the Deming improvement cycle of Plan-Do-Check-Act (PDCA) to software engineering. For example, EVO, which is one of the earliest agile methods, is known as a practical way to implement the PDCA cycle incrementally.

Many Agile methods are available in the literature. Some more popular approaches, discussed here briefly, include rapid application development (RAD), eXtreme programming (XP), Scrum, feature-driven development (FDD), and Lean software development.

- *RAD*: RAD methods are used primarily in data-intensive business systems application development. RAD is enabled by special-purpose database development tools used by software engineers to quickly develop, test and deploy new or modified business applications.
- *XP*: This approach uses stories or scenarios for requirements, develops tests first, has direct customer involvement on the team (typically defining acceptance tests), uses pair programming, and provides continuous code refactoring and integration. Stories are decomposed into tasks, prioritized, estimated, developed and tested. Each software increment is tested with automated and manual tests. An increment may be released frequently, such as every couple of weeks.
- *Scrum*: This Agile approach is more project management-friendly than the others. The Scrum master manages the activities within the project increment. Each increment is called a *sprint* and lasts no more than 30 days. The product owner determines which items go into the product backlog and develops a product

backlog item list. The tasks from this list are identified, defined, prioritized and estimated. A working version of the software is tested and released in each increment. Daily Scrum meetings ensure work is managed according to the plan.

- *FDD*: This is a model-driven, short, iterative software development approach using a five-phase process: (1) develop a product model to scope the breadth of the domain, (2) create the list of needs or features, (3) build the feature development plan, (4) develop designs for iteration-specific features, and (5) code, test, and then integrate the features. FDD is similar to an incremental software development approach. It is similar to XP, except that code ownership is assigned to individuals rather than to the team. In addition, FDD emphasizes an overall architectural approach to the software, which promotes building features correctly the first time rather than rely on continual refactoring.
- *Lean*: This is an application of lean manufacturing principles adapted from the Toyota Production System to software development. The approach adopts the strategy of making a Minimum Viable Product, in which a team releases the simplest version of its product. The team learns feedback from users and iterates based on the feedback. The concept of Lean is to optimize the entire development process, rather than optimizing the individual development process. By overlooking the entire value flow, including design, manufacturing, sales, and service delivery, this approach optimizes the flow to quickly deliver the service to users. Kanban is also a lightweight process that applies many ideas of the Lean process. However, there are some fundamental differences because Kanban supports managing workflow and visualization.

There are many more variations of Agile methods in the literature and in practice. There will always be a place for heavyweight,

plan-based software engineering methods as well as places where Agile methods shine. In addition, new methods are arising from combinations of Agile and plan-based methods: Practitioners are defining these new methods to balance features from heavyweight and lightweight methods based primarily on organizational business needs. These business needs, as typically identified by project stakeholders, should and do drive the choice of software engineering method.

Large-scale and enterprise agile approaches reflect recent efforts to manage many agile teams and apply agile principles and practices across the enterprise while keeping promises of agile development methodologies (see agile models in Software Engineering Process KA).

Agile methodology leads to shorter release cycles. Then, release engineering contributes a

lightweight release cycle. Release engineering is a sub-discipline in software engineering concerned with the compilation, assembly, and delivery of source code into finished products or other software components. The trend cycle in Agile would be Integration, Building, and Testing that Release Engineering focuses on. DevOps is often conflated with agile and continuous deployment approaches of software development. To avoid conflating, release management acts as a method for filling the collaboration gap between development and operations. Release managers need to monitor the development process and the promotion schedule for each release. The key to managing software releases in DevOps that keeps pace with deployment schedules is through automated management tools such as a continuous integration (CI) system.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	Budgen 2021 [1*]	Mellor and Balcer 2002 [2*]	Sommerville 2016 [3*]	Page-Jones 1999 [4*]	Wing 1990 [5*]	Brookshear 2008 [6*]{Brookshear, 2008, Computer Science: An Overview}	Boehm and Turner 2003 [7*]
1. Modeling							
<i>1.1. Modeling Principles</i>	c3s3, c3s5, c4s2, c7s1, c7s2	c2s2	c5s0				
<i>1.2. Properties and Expression of Models</i>	c7s2, c7s3		c4s1.1p7, c4s6p3, c5s0p3				
<i>1.3. Syntax, Semantics and Pragmatics</i>		c2s2.2.2p6	c5s0				
<i>1.4. Preconditions, Postconditions and Invariants</i>		c4s4		c10s4p2, c10s5p2p4			

2. Types of Models							
2.1. <i>Structural Modeling</i>	c9s5, c10s5		c8s1, c5s3	c4			
2.2. <i>Behavioral Modeling</i>	c9s3, c10s6	c9s2	c5s4				
3. Analysis of Models							
3.1. <i>Analyzing for Completeness</i>			c4s1.1p7, c4s6		pp8-11		
3.2. <i>Analyzing for Consistency</i>			c4s1.1p7, c4s6		pp8-11		
3.3. <i>Analyzing for Correctness</i>					pp8-11		
3.4. <i>Traceability</i>			c4s7.1, c4s7.2				
3.5. <i>Interaction Analysis</i>		c10, c11	c29s1.1, c29s5	c5			
4. Software Engineering Methods							
4.1. <i>Heuristic Methods</i>	c13		c2s2.2, c7s1, c5s4.1				
4.2. <i>Formal Methods</i>	c18s2		c27		pp8-24		
4.3. <i>Prototyping Methods</i>	c14s1, c14s2, c14s3		c2s3.1			c7s3p5	
4.4. <i>Agile Methods</i>	c14s5, c14s6		c3			c7s3p7	c6, app. A

REFERENCES

- [1*] D. Budgen, *Software Design: Creating Solutions for Ill-Structured Problems*, 3rd Edition, CRC Press, 2021.
- [2*] S.J. Mellor and M.J. Balcer, *Executable UML: A Foundation for Model-Driven Architecture*, 1st ed. Addison-Wesley, 2002.
- [3*] I. Sommerville, *Software Engineering*, 10th ed. Addison-Wesley, 2016.
- [4*] M. Page-Jones, *Fundamentals of Object-Oriented Design in UML*, 1st ed. Addison-Wesley, 1999.
- [5*] J.M. Wing, "A Specifier's Introduction to Formal Methods," *Computer*, vol. 23, pp. 8, 10-23, 1990.
- [6*] J.G. Brookshear, *Computer Science: An Overview*, 10th ed. Addison-Wesley, 2008.
- [7*] B. Boehm and R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley, 2003.
- [8] B. Selic and S. Gerard, *Modeling and Analysis of Real-Time and Embedded Systems with UML and MARTE: Developing Cyber-Physical Systems*, Morgan Kaufmann, 2013.

- [9] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*, Morgan & Claypool Publishers, 2017.
- [10] D. Jackson, *Software Abstractions*, revised edition, The MIT Press, 2016.
- [11] R. Aarenstrup, *Managing Model-Based Design*, CreateSpace Independent Publishing Platform, 2015.
- [12] C. Larman, *Applying UML and Patterns: An Introduction to Object-oriented Analysis and Design and Iterative Development*, Prentice Hall PTR, 2005.
- [13] M. Poppendieck and T. Poppendieck, *Lean Software Development: An Agile Toolkit*, Addison-Wesley Professional, 2003.
- [14] T. Ohno, *Toyota Production System: Beyond Large-Scale Production*, Taylor & Francis Distribution, 2021.
- [15] D.J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*, Blue Hole Press; 2010.
- [16] J. Goodpasture, *Project management the agile way: Making it work in the enterprise*, J. Ross Publishing, 2010.
- [17] ISO/IEC 19505-1:2012, Information technology — Object Management Group Unified Modeling Language (OMG UML) — Part 1: Infrastructure.
- [18] ISO/IEC/IEEE 32675:2022, Information technology — DevOps — Building reliable and secure systems including application build, package and deployment.
- [19] G. Kiczales, J. Lamping, A. Mendhekar, C. Maeda, C. Lopes, J. M. Loingtier, and J. Irwin, Aspect-oriented programming, ECOOP'97, LNCS, Vol. 1241, 1997.